

SBA301 – LAB 02

CREATE SINGLE PAGE APPLICATION WITH REACTJS

ORCHID GALLERY SPA – FETCHING AND CACHING DATA

Hướng dẫn thực hành chi tiết • React Bootstrap • Modal • Service Layer • Fetch/Axios • Cache

Môn học	SBA301 – Integrate Single Page Application with Spring Boot
Liên hệ kiến thức	Slot 6 Hooks → Slot 7 Routing → Slot 8 HTTP/REST → Slot 9 Fetching/Caching
Lab	Lab 02 – Create Single Page Application with ReactJS
Mức độ	Trung bình
Thời lượng	3–5 giờ thực hành
Công cụ	IntelliJ IDEA hoặc VS Code, Chrome/Edge DevTools
Sản phẩm	Source code + README + ảnh UI/DevTools + evidence checklist

Mục tiêu của tài liệu: giúp sinh viên hoàn thành đúng yêu cầu lõi của Lab 02 trước, sau đó nâng cấp bài Lab bằng kiến thức Slot 9 theo một lộ trình có kiểm soát. Phần mở rộng không thay thế yêu cầu gốc của Lab 02.

CÁCH SỬ DỤNG TÀI LIỆU

Tài liệu chia Lab 02 thành hai lớp. Lớp A là phần bắt buộc tương thích Lab 02 gốc: React Bootstrap, NavBar, danh sách Orchids, Card và Modal. Lớp B là phần nâng cấp bằng Slot 9: service layer, Promise/async-await, Fetch/Axios, loading/error/data, cache và DevTools. Sinh viên phải hoàn thành Lớp A trước khi chuyển sang Lớp B.

Lớp	Mục tiêu	Phạm vi
A – Lab 02 Core	Đáp ứng yêu cầu UI/component của Lab 02	React Bootstrap, NavBar, OrchidsData, Card, Modal
B – Slot 9 Extension	Biến Lab tĩnh thành ứng dụng có data flow thực tế hơn	Promise, async/await, service, Fetch/Axios, state, cache, DevTools
C – Optional	Mở rộng sau khi core đã PASS	Search/filter, json-server, Router, React Query/SWR awareness

Quy tắc Human Gate: AI có thể đề xuất code, nhưng sinh viên phải chạy, trace data flow, giải thích state transition và cung cấp evidence trước khi đánh dấu PASS.

0. THÔNG TIN DỰ ÁN

Thuộc tính	Nội dung
Tên dự án	Orchid Gallery SPA – Fetch and Cache Demo
Slot liên quan	Slot 9 – Fetching and Caching Data
Lab liên quan	Lab 02 – Create Single Page Application with ReactJS
Mức độ	Trung bình
Thời lượng	3–5 giờ
Hình thức	Cá nhân hoặc nhóm nhỏ
Frontend	ReactJS + Vite + React Bootstrap
Data	Static JS → Promise mock → public/orchids.json → optional Axios/json-server
IDE	IntelliJ IDEA hoặc VS Code
Evidence	UI screenshots, Network screenshot, console/terminal, Git commit, README

1. TÓM TẮT DỰ ÁN VÀ MỤC TIÊU

1.1 Dự án làm gì?

- Xây dựng ứng dụng React hiển thị danh sách Orchids bằng React Bootstrap Card.
- Tạo NavBar và Modal Detail theo yêu cầu Lab 02.
- Quản lý selectedOrchid và trạng thái mở/đóng Modal bằng useState.
- Tách logic lấy dữ liệu sang orchidService và custom hook useOrchids.
- Thực hành Promise, async/await, Fetch hoặc Axios, cùng loading/error/data state.
- Thực hành cache đơn giản có TTL và cơ chế Force Reload bỏ cache.
- Quan sát request /orchids.json bằng DevTools Network và giải thích data flow.

1.2 Mục tiêu kiến thức và kỹ năng

Nhóm	Sinh viên phải làm được
React/Lab 02	Tạo component, props, state, list rendering, React Bootstrap, Modal.

Nhóm	Sinh viên phải làm được
Asynchronous	Giải thích Promise, async/await; biết try/catch/finally.
HTTP client	Gọi dữ liệu bằng Fetch; biết khác biệt cơ bản với Axios.
State model	Phân biệt loading, error, empty, data.
Architecture	Tách UI khỏi data access bằng service layer và hook.
Caching	Giải thích cache hit, cache miss, TTL và stale data.
Debug	Dùng Console + Network để phân tích request, response, 404 và JSON lỗi.

2. CHỨC NĂNG CỦA ỨNG DỤNG

Mã/Chức năng	Mô tả	Kiến thức
F01 NavBar	Tên Orchid Gallery; các mục Home/Orchids/About ở mức giao diện.	React Bootstrap NavBar
F02 Orchid list	Danh sách Orchid dạng responsive Card grid.	map(), props, Card/Container/Row/Col
F03 Orchid detail	Bấm Detail → selectedOrchid → Modal mở.	useState, event, Modal
F04 Loading	Spinner khi dữ liệu đang tải.	loading state
F05 Error	Alert lỗi + Try Again.	error state, retry
F06 Empty	Thông báo khi danh sách rỗng.	derived UI state
F07 Service layer	orchidService.getOrchids().	separation of concerns
F08 Promise mock	Delay giả lập API trước khi dùng HTTP thật.	Promise, async/await
F09 Fetch JSON	GET /orchids.json.	Fetch, response.ok, JSON
F10 Axios awareness	Có phiên bản service bằng Axios để so sánh.	Axios response.data
F11 Cache	Module cache TTL 30s; force reload invalidates.	client-side caching
F12 DevTools evidence	Quan sát request, status, type, timing.	Network tab
F13 Search/filter optional	Tên, category, isSpecial.	derived data; không gọi API thêm
F14 README/evidence	Hướng dẫn chạy + ảnh chứng minh.	documentation

3. CÔNG NGHỆ VÀ CÔNG CỤ SỬ DỤNG

Công nghệ/Công cụ	Vai trò	Ghi chú
Node.js	Chạy npm và Vite	Syllabus dùng Node 21; xem lưu ý tương thích Vite bên dưới.
Vite	Scaffold + dev server	Node 21: pin Vite 5; Node 22.12+: có thể dùng Vite hiện hành.
ReactJS	UI và component model	Function components + Hooks.

Công nghệ/Công cụ	Vai trò	Ghi chú
React Bootstrap	Navbar, Card, Modal, Alert, Spinner, Form	Cài cùng bootstrap CSS.
Fetch API	HTTP client có sẵn trong browser	Core extension.
Axios	HTTP client library	Comparison/extension.
Browser DevTools	Inspect request/response/cache	Evidence bắt buộc khi dùng Fetch.
IntelliJ/VS Code	IDE	Terminal, edit, run, inspect.
Git	Checkpoint và evidence	Khuyến nghị commit theo milestone.

LƯU Ý TƯƠNG THÍCH: Vite hiện hành yêu cầu Node 20.19+ hoặc Node 22.12+; Node 21 không nằm trong dải hỗ trợ. Nếu phòng máy đang khóa Node 21, tài liệu dùng create-vite/Vite 5 để tránh lỗi môi trường. Nếu có Node 22.12+ thì có thể dùng @latest.

4. CẤU TRÚC THƯ MỤC DỰ ÁN

```
orchid-gallery-spa/  
├── public/  
│   ├── orchids.json  
│   └── images/  
│       └── orchid-placeholder.svg  
├── src/  
│   ├── api/  
│   │   ├── apiClient.js  
│   │   └── orchidService.js  
│   ├── components/  
│   │   ├── NavBar.jsx  
│   │   ├── Orchids.jsx  
│   │   ├── OrchidCard.jsx  
│   │   ├── OrchidDetailModal.jsx  
│   │   ├── SearchBox.jsx  
│   │   ├── CategoryFilter.jsx  
│   │   ├── LoadingSpinner.jsx  
│   │   └── ErrorMessage.jsx  
│   ├── hooks/  
│   │   └── useOrchids.js  
│   ├── shared/  
│   │   └── ListOfOrchids.js  
│   ├── styles/  
│   │   └── app.css  
│   ├── App.jsx  
│   └── main.jsx  
├── README.md  
├── package.json  
└── vite.config.js
```

Thư mục	Vai trò
components	UI components. Không chứa logic HTTP chi tiết.
shared	Dữ liệu tĩnh ban đầu để hoàn thành Lab 02 core.
api	Data-access layer. Tập trung Fetch/Axios/cache.
hooks	Kết nối service với React state/lifecycle.

Thư mục	Vai trò
public	Static assets và orchids.json truy cập trực tiếp qua URL.
styles	CSS tùy chỉnh ngoài Bootstrap.

5. YÊU CẦU CÀI ĐẶT VÀ CHUẨN BỊ MÔI TRƯỜNG

5.1 Kiểm tra Node/npm

```
node -v
npm -v
npm --version
```

Nếu Node = 21.x, không dùng `npm create vite@latest`; dùng lệnh Vite 5 ở bước tạo project. Nếu Node >= 22.12, có thể dùng Vite hiện hành.

5.2 Tạo project

Nhánh A – máy phòng lab dùng Node 21 (khuyến nghị theo syllabus):

```
npm create vite@5 orchid-gallery-spa -- --template react
cd orchid-gallery-spa
npm install
```

Nhánh B – máy dùng Node 22.12+ hoặc phiên bản hiện hành tương thích:

```
npm create vite@latest orchid-gallery-spa -- --template react
cd orchid-gallery-spa
npm install
```

5.3 Cài thư viện

```
npm install react-bootstrap bootstrap
npm install axios
```

React-Bootstrap yêu cầu stylesheet Bootstrap được import riêng; bản thân React-Bootstrap không tự mang CSS vào ứng dụng.

5.4 IntelliJ

1. File > Open → chọn thư mục orchid-gallery-spa.
2. Mở Terminal: Alt + F12 hoặc View > Tool Windows > Terminal.
3. Mở package.json và kiểm tra scripts dev/build.
4. Có thể tạo Run > Edit Configurations > npm > script dev.
5. Nếu IDE edition không hỗ trợ JavaScript/React đầy đủ, vẫn dùng Terminal hoặc mở cùng project bằng VS Code.

5.5 Smoke test

```
npm run dev
```

PASS khi terminal không có lỗi blocking, URL dev server mở được và trang React mặc định hiển thị.

6. TÓM TẮT DANH SÁCH CÁC BƯỚC THỰC HIỆN

#	Bước	Milestone
1	Tạo project Vite phù hợp phiên bản Node	V0
2	Mở project trong IntelliJ/VS Code và chạy baseline	V0
3	Cài React Bootstrap, Bootstrap và Axios	V1
4	Import Bootstrap CSS	V1
5	Dọn code Vite mặc định	V1
6	Tạo cấu trúc thư mục	V2

#	Bước	Milestone
7	Tạo local placeholder image	V2
8	Tạo ListOfOrchids.js với 8 Orchid	V2
9	Tạo NavBar.jsx	V3
10	Tạo OrchidCard.jsx	V3
11	Tạo OrchidDetailModal.jsx	V3
12	Tạo Orchids.jsx từ dữ liệu tĩnh	V4
13	Ghép App.jsx và kiểm tra Lab 02 Core	V4
14	Tạo LoadingSpinner/ErrorMessage	V4
15	Tạo Promise-based orchidService	V5
16	Tạo useOrchids custom hook	V5
17	Chuyển Orchids sang loading/error/data flow	V5
18	Tạo public/orchids.json	V6
19	Chuyển orchidService sang Fetch	V6
20	Dùng DevTools kiểm tra request	V6
21	Thêm cache TTL và Force Reload	V7
22	Tạo phiên bản Axios để so sánh	V7
23	Thêm search/category/special filter (optional)	V7
24	Kiểm tra lỗi có chủ đích	V8
25	Build production + Git checkpoint	V8
26	README + Evidence Package + Human Gate	V8

7. HƯỚNG DẪN CHI TIẾT TỪNG BƯỚC

Bước 1. Tạo project React bằng Vite

Trạng thái sau bước: V0 – Project được scaffold

Mục	Nội dung
Mục tiêu	Có project React tối thiểu chạy được.
Thao tác	Mở Terminal ở thư mục workspace; chọn đúng lệnh theo phiên bản Node ở Phần 5.
Kết quả mong đợi	Thư mục orchid-gallery-spa được tạo và npm install hoàn tất.
Cách kiểm tra	Chạy npm run dev; mở URL Vite.
Lỗi thường gặp	Node 21 + Vite latest báo unsupported engine → dùng Vite 5 hoặc nâng Node.

Bước 2. Mở project trong IntelliJ/VS Code và chạy baseline

Trạng thái sau bước: V0 – Baseline verified

Mục	Nội dung
Mục tiêu	Xác nhận IDE và dev server hoạt động.
Thao tác	File > Open; chọn project; mở Terminal; chạy npm run dev.
Kết quả mong đợi	Ứng dụng mặc định hiển thị; Console không có lỗi blocking.
Cách kiểm tra	Sửa tạm tiêu đề trong App.jsx để kiểm tra hot reload.
Lỗi thường gặp	Mở nhầm thư mục cha hoặc terminal không ở project root.

Bước 3. Cài React Bootstrap, Bootstrap và Axios

Trạng thái sau bước: V1 – Dependencies ready

```
npm install react-bootstrap bootstrap
```

```
npm install axios
```

Mục	Nội dung
Mục tiêu	Có UI library và HTTP client mở rộng.
Thao tác	Chạy hai lệnh trên; kiểm tra package.json dependencies.
Kết quả mong đợi	react-bootstrap, bootstrap và axios xuất hiện trong package.json.
Cách kiểm tra	npm ls react-bootstrap bootstrap axios.
Lỗi thường gặp	Network/proxy npm hoặc package-lock lỗi; thử npm install lại sau khi xác nhận mạng.

Bước 4. Import Bootstrap CSS

Trạng thái sau bước: V1 – Bootstrap styling active

```
// src/main.jsx
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import 'bootstrap/dist/css/bootstrap.min.css';
import './styles/app.css';
import App from './App.jsx';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

Mục	Nội dung
Mục tiêu	Bootstrap CSS có hiệu lực toàn app.
Thao tác	Sửa src/main.jsx như trên.
Kết quả mong đợi	Component Bootstrap sau này hiển thị đúng style.
Cách kiểm tra	Tạo Button tạm nếu cần; inspect class .btn.
Lỗi thường gặp	Import CSS sai path hoặc import file app.css chưa tồn tại → tạo file ở bước

Mục	Nội dung
	tiếp theo.

Bước 5. Dọn code mặc định và tạo CSS nền

Trạng thái sau bước: V1 – Clean baseline

```
/* src/styles/app.css */
body {
  margin: 0;
  background: #f8f9fa;
  color: #212529;
}

.orchid-card-img {
  height: 220px;
  object-fit: cover;
}

.orchid-card {
  height: 100%;
}
```

Mục	Nội dung
Mục tiêu	Loại bỏ logo/style mẫu Vite và có CSS tối thiểu.
Thao tác	Tạo src/styles/app.css; xóa import App.css cũ nếu không dùng.
Kết quả mong đợi	Giao diện nền sạch, không còn Vite demo.
Cách kiểm tra	Reload trang, xem Console.
Lỗi thường gặp	main.jsx import app.css trước khi file được tạo.

Bước 6. Tạo cấu trúc thư mục

Trạng thái sau bước: V2 – Architecture skeleton

Mục	Nội dung
Mục tiêu	Chuẩn bị đúng separation of concerns.
Thao tác	Tạo src/components, src/shared, src/api, src/hooks, src/styles; public/images.
Kết quả mong đợi	Project tree đúng Phần 4.
Cách kiểm tra	Mở Project tool window và đối chiếu.
Lỗi thường gặp	Tên thư mục api/API khác case dẫn tới import lỗi trên Linux.

Bước 7. Tạo local placeholder image

Trạng thái sau bước: V2 – Image asset ready

```
<!-- public/images/orchid-placeholder.svg -->
<svg xmlns="http://www.w3.org/2000/svg" width="640" height="420" viewBox="0 0 640 420">
  <rect width="640" height="420" fill="#eef3f7"/>
  <circle cx="320" cy="185" r="90" fill="#d5b3e6"/>
  <circle cx="250" cy="150" r="55" fill="#efd5f5"/>
  <circle cx="390" cy="150" r="55" fill="#efd5f5"/>
```



```
<text x="320" y="340" text-anchor="middle" font-family="Arial" font-size="28" fill="#495057">Orchid</text>
</svg>
```

Trong Vite, file đặt trong `public/images/...` được dùng bằng browser path `/images/...`, không viết `/public/images/...`.

Mục	Nội dung
Mục tiêu	Không phụ thuộc Internet để hiện ảnh.
Thao tác	Tạo file SVG trên; có thể thay bằng ảnh Lab 02 chính thức nếu giảng viên cung cấp.
Kết quả mong đợi	Mở /images/orchid-placeholder.svg thấy ảnh.
Cách kiểm tra	Truy cập trực tiếp URL asset trên browser.
Lỗi thường gặp	Dùng src="public/images/..." gây 404.

Bước 8. Tạo dữ liệu OrchidsData với 8 bản ghi

Trạng thái sau bước: V3 – Static data ready

```
// src/shared/ListOfOrchids.js
export const OrchidsData = [
  { id: '1', orchidName: 'Ceasar 4N', category: 'Dendrobium', isSpecial: true, origin: 'Thailand', color: 'Purple', rating: 4.8, image: '/images/orchid-placeholder.svg', description: 'Dendrobium nổi bật với màu tím và form hoa cân đối.' },
  { id: '2', orchidName: 'Mini Yaya', category: 'Dendrobium', isSpecial: true, origin: 'Taiwan', color: 'Pink', rating: 4.6, image: '/images/orchid-placeholder.svg', description: 'Giống mini nhỏ gọn, phù hợp không gian trưng bày.' },
  { id: '3', orchidName: 'Crystallinum', category: 'Dendrobium', isSpecial: true, origin: 'Southeast Asia', color: 'White', rating: 4.7, image: '/images/orchid-placeholder.svg', description: 'Hoa sáng màu, cánh thanh và có tính trang trí cao.' },
  { id: '4', orchidName: 'Damari', category: 'Dendrobium', isSpecial: false, origin: 'Hybrid', color: 'Yellow', rating: 4.2, image: '/images/orchid-placeholder.svg', description: 'Giống lai dễ quan sát đặc điểm hình thái.' },
  { id: '5', orchidName: 'Taichung Beauty', category: 'Cattleya', isSpecial: true, origin: 'Taiwan', color: 'Violet', rating: 4.9, image: '/images/orchid-placeholder.svg', description: 'Cattleya có hoa lớn, màu sắc nổi bật.' },
  { id: '6', orchidName: 'Snow White', category: 'Phalaenopsis', isSpecial: false, origin: 'Hybrid', color: 'White', rating: 4.3, image: '/images/orchid-placeholder.svg', description: 'Phalaenopsis trắng, phù hợp minh họa category khác.' },
  { id: '7', orchidName: 'Golden Shower', category: 'Oncidium', isSpecial: false, origin: 'Hybrid', color: 'Yellow', rating: 4.4, image: '/images/orchid-placeholder.svg', description: 'Chùm hoa vàng nhỏ, tạo khác biệt khi lọc category.' },
  { id: '8', orchidName: 'Blue Vanda', category: 'Vanda', isSpecial: true, origin: 'Asia', color: 'Blue', rating: 4.8, image: '/images/orchid-placeholder.svg', description: 'Vanda màu xanh tím, dùng để kiểm tra filter và detail.' }
];
```

Mục	Nội dung
Mục tiêu	Có dữ liệu đủ để test list/filter/modal.
Thao tác	Tạo file đúng path; export named `OrchidsData`.
Kết quả mong đợi	Import không lỗi; 8 id duy nhất.
Cách kiểm tra	Tạm console.log(OrchidsData.length) = 8.
Lỗi thường gặp	Sai export default/named import; id trùng gây key warning.

Bước 9. Tạo NavBar.jsx

Trạng thái sau bước: V3 – Lab 02 Activity 01 complete

```
// src/components/NavBar.jsx
import { Container, Nav, Navbar } from 'react-bootstrap';

export default function NavBar() {
  return (
    <Navbar bg="primary" data-bs-theme="dark" expand="lg">
      <Container>
        <Navbar.Brand href="#">Orchid Gallery</Navbar.Brand>
```

```
<Navbar.Toggle aria-controls="orchid-navbar" />
<Navbar.Collapse id="orchid-navbar">
  <Nav className="me-auto">
    <Nav.Link href="#home">Home</Nav.Link>
    <Nav.Link href="#orchids">Orchids</Nav.Link>
    <Nav.Link href="#about">About</Nav.Link>
  </Nav>
</Navbar.Collapse>
</Container>
</Navbar>
);
}
```

Mục	Nội dung
Mục tiêu	Đáp ứng Activity 01 của Lab 02.
Thao tác	Tạo file và import đúng React-Bootstrap components.
Kết quả mong đợi	Navbar responsive hiển thị, có brand và menu.
Cách kiểm tra	Resize browser để kiểm tra Toggle.
Lỗi thường gặp	Chưa import Bootstrap CSS → Navbar không được style.

Bước 10. Tạo OrchidCard.jsx

Trạng thái sau bước: V4 – Reusable Card

```
// src/components/OrchidCard.jsx
import { Badge, Button, Card } from 'react-bootstrap';

export default function OrchidCard({ orchid, onDetail }) {
  return (
    <Card className="orchid-card shadow-sm">
      <Card.Img className="orchid-card-img" variant="top" src={orchid.image} alt={orchid.orchidName} />
      <Card.Body className="d-flex flex-column">
        <div className="d-flex justify-content-between align-items-start gap-2">
          <Card.Title>{orchid.orchidName}</Card.Title>
          {orchid.isSpecial && <Badge bg="warning" text="dark">Special</Badge>}
        </div>
        <Card.Text className="text-muted">{orchid.category}</Card.Text>
        <Button className="mt-auto" onClick={() => onDetail(orchid)}>Detail</Button>
      </Card.Body>
    </Card>
  );
}
```

Mục	Nội dung
Mục tiêu	Card nhận dữ liệu qua props và phát event lên parent.
Thao tác	Tạo component như code.
Kết quả mong đợi	Card không tự quản lý selected orchid; callback onDetail được gọi khi click.
Cách kiểm tra	React DevTools/console hoặc test bằng parent ở bước 12.
Lỗi thường gặp	Gọi onDetail() ngay trong render thay vì trong arrow function.

Bước 11. Tạo OrchidDetailModal.jsx

Trạng thái sau bước: V5 – Detail component ready

```
// src/components/OrchidDetailModal.jsx
import { Badge, Button, Modal } from 'react-bootstrap';

export default function OrchidDetailModal({ show, orchid, onClose }) {
  return (
    <Modal show={show} onHide={onClose} centered>
      <Modal.Header closeButton>
        <Modal.Title>{orchid?.orchidName ?? 'Orchid detail'}</Modal.Title>
      </Modal.Header>
      <Modal.Body>
        {orchid ? (
          <>
            <img src={orchid.image} alt={orchid.orchidName} className="img-fluid rounded mb-3 w-100" />
            <p><strong>Category:</strong> {orchid.category}</p>
            <p><strong>Origin:</strong> {orchid.origin}</p>
            <p><strong>Color:</strong> {orchid.color}</p>
            <p><strong>Rating:</strong> {orchid.rating}</p>
            <p><strong>Special:</strong> {orchid.isSpecial ? <Badge bg="warning" text="dark">Yes</Badge> : 'No'}</p>
            <p>{orchid.description}</p>
          </>
        ) : <p>Chưa chọn Orchid.</p>}
      </Modal.Body>
      <Modal.Footer>
        <Button variant="secondary" onClick={onClose}>Close</Button>
      </Modal.Footer>
    </Modal>
  );
}
```

Mục	Nội dung
Mục tiêu	Modal an toàn khi orchid=null.
Thao tác	Tạo file như trên.
Kết quả mong đợi	Modal nhận state từ parent; nút X và Close đều đóng.
Cách kiểm tra	Test ở bước 12.
Lỗi thường gặp	Render orchid.orchidName khi orchid null → crash; dùng optional chaining.

Bước 12. Tạo Orchids.jsx dùng dữ liệu tĩnh

Trạng thái sau bước: V5 – Lab 02 Core list + Modal complete

```
// src/components/Orchids.jsx
import { useState } from 'react';
import { Col, Container, Row } from 'react-bootstrap';
import { OrchidsData } from '../shared/ListOfOrchids';
import OrchidCard from './OrchidCard';
import OrchidDetailModal from './OrchidDetailModal';

export default function Orchids() {
  const [show, setShow] = useState(false);
  const [selectedOrchid, setSelectedOrchid] = useState(null);

  const handleShow = (orchid) => {
    setSelectedOrchid(orchid);
  }
}
```

```

    setShow(true);
  };

  const handleClose = () => {
    setShow(false);
    setSelectedOrchid(null);
  };

  return (
    <Container id="orchids" className="py-4">
      <h2 className="mb-3">Orchids List</h2>
      <Row>
        {OrchidsData.map((orchid) => (
          <Col xs={12} sm={6} lg={3} key={orchid.id} className="mb-4">
            <OrchidCard orchid={orchid} onDetail={handleShow} />
          </Col>
        ))}
      </Row>
      <OrchidDetailModal show={show} orchid={selectedOrchid} onClose={handleClose} />
    </Container>
  );
}

```

Mục	Nội dung
Mục tiêu	Hoàn thành Activity 02 + Modal theo Lab 02 core.
Thao tác	Tạo component; bảo đảm imports đúng.
Kết quả mong đợi	8 cards hiển thị; Detail mở đúng dữ liệu.
Cách kiểm tra	Bấm lần lượt 2 card khác nhau và kiểm tra Modal.
Lỗi thường gặp	Thiếu key; state đặt trong từng Card làm khó kiểm soát selectedOrchid.

Bước 13. Ghép App.jsx và xác nhận Lab 02 Core

Trạng thái sau bước: V5 – CORE PASS

```

// src/App.jsx
import NavBar from './components/NavBar';
import Orchids from './components/Orchids';

export default function App() {
  return (
    <>
      <NavBar />
      <main>
        <Orchids />
      </main>
    </>
  );
}

```

CORE PASS CRITERIA: NavBar hiển thị; danh sách Orchid bằng Card; mỗi Card có image/name/category/Detail; Modal hiển thị đúng Orchid và đóng được. Chỉ khi đạt 5 điều kiện này mới chuyển sang Slot 9 extension.

Mục	Nội dung
Mục tiêu	Đóng bằng baseline Lab 02 trước khi refactor data source.

Mục	Nội dung
Thao tác	Sửa App.jsx; chạy npm run dev.
Kết quả mong đợi	Lab 02 core hoạt động độc lập với network.
Cách kiểm tra	Chụp ảnh UI + Modal; commit Git `lab02-core-pass`.
Lỗi thường gặp	Refactor sang Fetch quá sớm khiến sinh viên khó phân biệt lỗi UI và lỗi data.

Bước 14. Tạo LoadingSpinner và ErrorMessage

Trạng thái sau bước: V6 – Async UI states ready

```
// src/components/LoadingSpinner.jsx
import { Spinner } from 'react-bootstrap';
export default function LoadingSpinner() {
  return <div className="text-center py-5"><Spinner animation="border" role="status" /><div className="mt-2">Loading Orchids...</div></div>;
}

// src/components/ErrorMessage.jsx
import { Alert, Button } from 'react-bootstrap';
export default function ErrorMessage({ message, onRetry }) {
  return <Alert variant="danger"><Alert.Heading>Không thể tải dữ liệu</Alert.Heading><p>{message}</p>{onRetry && <Button variant="outline-danger"
onClick={onRetry}>Try Again</Button></Alert>;
}
```

Mục	Nội dung
Mục tiêu	Chuẩn bị UI cho loading/error state.
Thao tác	Tạo hai file components.
Kết quả mong đợi	Có component tái sử dụng, không nhét markup lỗi/loading vào service.
Cách kiểm tra	Tạm render trong App để kiểm tra style.
Lỗi thường gặp	Không truyền onRetry nhưng component vẫn phải hoạt động.

Bước 15. Tạo orchidService giả lập Promise/setTimeout

Trạng thái sau bước: V6 – Promise service

```
// src/api/orchidService.js – PHIÊN BẢN A: MOCK PROMISE
import { OrchidsData } from '../shared/ListOfOrchids';

const orchidService = {
  async getOrchids() {
    await new Promise((resolve) => setTimeout(resolve, 700));
    return OrchidsData;
  },
  async getOrchidById(id) {
    await new Promise((resolve) => setTimeout(resolve, 300));
    const orchid = OrchidsData.find((item) => item.id === String(id));
    if (!orchid) throw new Error('Không tìm thấy Orchid');
    return orchid;
  }
};

export default orchidService;
```

Mục	Nội dung
-----	----------

Mục	Nội dung
Mục tiêu	Học async flow trước khi thêm HTTP.
Thao tác	Tạo service như code; chưa dùng Fetch.
Kết quả mong đợi	getOrchids() trả Promise và delay ~700ms.
Cách kiểm tra	Gọi tạm trong console/useEffect; quan sát delay.
Lỗi thường gặp	Quên return OrchidsData; so sánh id number/string không thống nhất.

Bước 16. Tạo custom hook useOrchids

Trạng thái sau bước: V7 – loading/error/data state model

```
// src/hooks/useOrchids.js
import { useCallback, useEffect, useState } from 'react';
import orchidService from '../api/orchidService';

export default function useOrchids() {
  const [orchids, setOrchids] = useState([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  const loadOrchids = useCallback(async () => {
    setLoading(true);
    setError(null);
    try {
      const data = await orchidService.getOrchids();
      setOrchids(data);
    } catch (err) {
      setError(err instanceof Error ? err.message : 'Không thể tải danh sách Orchids');
    } finally {
      setLoading(false);
    }
  }, []);

  useEffect(() => {
    loadOrchids();
  }, [loadOrchids]);

  return { orchids, loading, error, reload: loadOrchids };
}
```

Mục	Nội dung
Mục tiêu	Tách lifecycle/state khỏi UI.
Thao tác	Tạo hook và đọc từng dòng: start loading → try → set data → catch → finally.
Kết quả mong đợi	Hook trả API nhỏ: orchids/loading/error/reload.
Cách kiểm tra	React DevTools xem state đổi theo thời gian.
Lỗi thường gặp	Bỏ dependency array hoặc tạo function mới mỗi render có thể gây loop.

Bước 17. Chuyển Orchids.jsx sang service + state model

Trạng thái sau bước: V7 – Async UI complete

```
// src/components/Orchids.jsx – ASYNC VERSION
```

```
import { useState } from 'react';
import { Button, Col, Container, Row } from 'react-bootstrap';
import useOrchids from '../hooks/useOrchids';
import ErrorMessage from './ErrorMessage';
import LoadingSpinner from './LoadingSpinner';
import OrchidCard from './OrchidCard';
import OrchidDetailModal from './OrchidDetailModal';

export default function Orchids() {
  const { orchids, loading, error, reload } = useOrchids();
  const [show, setShow] = useState(false);
  const [selectedOrchid, setSelectedOrchid] = useState(null);

  const handleShow = (orchid) => { setSelectedOrchid(orchid); setShow(true); };
  const handleClose = () => { setShow(false); setSelectedOrchid(null); };

  return (
    <Container id="orchids" className="py-4">
      <div className="d-flex justify-content-between align-items-center mb-3">
        <h2 className="mb-0">Orchids List</h2>
        <Button variant="outline-primary" onClick={reload} disabled={loading}>Reload</Button>
      </div>
      {loading && <LoadingSpinner />}
      {error && <ErrorMessage message={error} onRetry={reload} />}
      {!loading && !error && orchids.length === 0 && <p>Không có Orchid nào.</p>}
      {!loading && !error && orchids.length > 0 && {
        <Row>
          {orchids.map((orchid) => <Col xs={12} sm={6} lg={3} key={orchid.id} className="mb-4"><OrchidCard orchid={orchid} onDetail={handleShow} /></Col>)}
        </Row>
      )}
      <OrchidDetailModal show={show} orchid={selectedOrchid} onClose={handleClose} />
    </Container>
  );
}
```

Mục	Nội dung
Mục tiêu	UI không biết dữ liệu đến từ đâu.
Thao tác	Thay Orchids.jsx bằng phiên bản async.
Kết quả mong đợi	Thấy Spinner ~700ms rồi list; Reload lặp flow.
Cách kiểm tra	Quan sát state transitions.
Lỗi thường gặp	Render list đồng thời loading gây UI nhấp nháy; dùng điều kiện rõ ràng.

Bước 18. Tạo public/orchids.json

Trạng thái sau bước: V8 – HTTP-readable data

```
[
  {
    "id": "1",
    "orchidName": "Ceasar 4N",
    "category": "Dendrobium",
    "isSpecial": true,
    "origin": "Thailand",
    "color": "Purple",
    "rating": 4.8,
    "image": "/images/orchid-placeholder.svg",
    "description": "Dendrobium nổi bật với màu tím và form hoa cân đối."
  },
  {
    "id": "2",
    "orchidName": "Mini Yaya",
    "category": "Dendrobium",
    "isSpecial": true,
    "origin": "Taiwan",
    "color": "Pink",
    "rating": 4.6,
    "image": "/images/orchid-placeholder.svg",
    "description": "Giống mini nhỏ gọn, phù hợp không gian trưng bày."
  },
  {
    "id": "3",
    "orchidName": "Crystallinum",
    "category": "Dendrobium",
    "isSpecial": true,
    "origin": "Southeast Asia",
    "color": "White",
    "rating": 4.7,
    "image": "/images/orchid-placeholder.svg",
    "description": "Hoa sáng màu, cánh thanh và có tính trang trí cao."
  },
  {
    "id": "4",
    "orchidName": "Damari",
    "category": "Dendrobium",
    "isSpecial": false,
    "origin": "Hybrid",
    "color": "Yellow",
    "rating": 4.2,
    "image": "/images/orchid-placeholder.svg",
    "description": "Giống lai dễ quan sát đặc điểm hình thái."
  },
  {
    "id": "5",
    "orchidName": "Taichung Beauty",
    "category": "Cattleya",
    "isSpecial": true,
    "origin": "Taiwan",
    "color": "Violet",
    "rating": 4.9,
    "image": "/images/orchid-placeholder.svg",
    "description": "Cattleya có hoa lớn, màu sắc nổi bật."
  }
]
```

```
{
  "id": "6",
  "orchidName": "Snow White",
  "category": "Phalaenopsis",
  "isSpecial": false,
  "origin": "Hybrid",
  "color": "White",
  "rating": 4.3,
  "image": "/images/orchid-placeholder.svg",
  "description": "Phalaenopsis trắng, phù hợp minh họa category khác."
},
{
  "id": "7",
  "orchidName": "Golden Shower",
  "category": "Oncidium",
  "isSpecial": false,
  "origin": "Hybrid",
  "color": "Yellow",
  "rating": 4.4,
  "image": "/images/orchid-placeholder.svg",
  "description": "Chùm hoa vàng nhỏ, tạo khác biệt khi lọc category."
},
{
  "id": "8",
  "orchidName": "Blue Vanda",
  "category": "Vanda",
  "isSpecial": true,
  "origin": "Asia",
  "color": "Blue",
  "rating": 4.8,
  "image": "/images/orchid-placeholder.svg",
  "description": "Vanda màu xanh tím, dùng để kiểm tra filter và detail."
}
]
```

Mục	Nội dung
Mục tiêu	Chuyển JS objects thành JSON hợp lệ.
Thao tác	Tạo public/orchids.json; không có export, comment hoặc trailing comma.
Kết quả mong đợi	Mở /orchids.json thấy JSON array.
Cách kiểm tra	Browser trực tiếp + DevTools Network.
Lỗi thường gặp	Nhầm JS object với JSON: key không quote, single quote, trailing comma.

Bước 19. Chuyển orchidService sang Fetch

Trạng thái sau bước: V8 – Fetch integration

```
// src/api/orchidService.js – PHIÊN BẢN B: FETCH
const orchidService = {
  async getOrchids() {
    const response = await fetch('/orchids.json', { headers: { Accept: 'application/json' } });
    if (!response.ok) throw new Error(`HTTP ${response.status}: Không thể tải orchids.json`);
    return response.json();
  }
};
export default orchidService;
```

Điểm phải giải thích: fetch chỉ reject khi lỗi mạng; HTTP 404/500 vẫn cần kiểm tra `response.ok` hoặc `response.status`.

Mục	Nội dung
Mục tiêu	Thay mock Promise bằng HTTP GET thật từ dev server.
Thao tác	Thay nội dung service. Không sửa UI/hook.
Kết quả mong đợi	List vẫn hiển thị; Network có request /orchids.json.
Cách kiểm tra	DevTools → Network → orchids.json → Headers/Response.
Lỗi thường gặp	File sai path, JSON invalid, không check response.ok.

Bước 20. Quan sát request/response bằng DevTools

Trạng thái sau bước: V8 – Network evidence

- F12 mở DevTools.
- Chọn Network; bật Preserve log nếu muốn.
- Reload trang.
- Chọn request `orchids.json`.
- Ghi Request URL, Method, Status Code, Response Headers, Content-Type, Response body, Timing.
- Chụp ảnh làm evidence.

Trường cần đọc	Giá trị mong đợi
Request URL	http://localhost:5173/orchids.json
Method	GET

Trường cần đọc	Giá trị mong đợi
Status	200
Content-Type	application/json hoặc JSON-compatible
Response	Mảng 8 Orchid
Initiator	fetch từ orchidService/useOrchids flow
Mục	Nội dung
Mục tiêu	Sinh viên thấy HTTP ở mức thực tế.
Thao tác	Thực hiện 6 thao tác trên.
Kết quả mong đợi	Có screenshot Network đủ thông tin.
Cách kiểm tra	Giải thích bằng lời: component → hook → service → fetch → response → state → render.
Lỗi thường gặp	Chỉ nhìn Response mà không đọc status/headers/timing.

Bước 21. Thêm cache TTL và Force Reload

Trạng thái sau bước: V10 – Simple cache

```
// src/api/orchidService.js – FETCH + CACHE
let orchidCache = null;
let cacheTime = 0;
const CACHE_DURATION = 30_000;

async function fetchOrchids() {
  const response = await fetch('/orchids.json', { headers: { Accept: 'application/json' } });
  if (!response.ok) throw new Error(`HTTP ${response.status}: Không thể tải orchids.json`);
  return response.json();
}

const orchidService = {
  async getOrchids({ force = false } = {}) {
    const now = Date.now();
    const validCache = orchidCache && (now - cacheTime < CACHE_DURATION);
    if (!force && validCache) return orchidCache;
    const data = await fetchOrchids();
    orchidCache = data;
    cacheTime = now;
    return data;
  },
  clearCache() { orchidCache = null; cacheTime = 0; }
};
export default orchidService;

// cập nhật src/hooks/useOrchids.js – phần loadOrchids
const loadOrchids = useCallback(async (force = false) => {
  setLoading(true);
  setError(null);
  try {
    const data = await orchidService.getOrchids({ force });
    setOrchids(data);
  } catch (err) {
    setError(err instanceof Error ? err.message : 'Không thể tải danh sách Orchids');
  }
});
```

```
} finally {  
  setLoading(false);  
}  
}, []);
```

```
const reload = useCallback(() => loadOrchids(true), [loadOrchids]);
```

Sau chỉnh sửa trên, return của hook phải là `{ orchids, loading, error, reload }`, và `useEffect` vẫn gọi `loadOrchids(false)`.

Mục	Nội dung
Mục tiêu	Phân biệt cache hit và force refresh.
Thao tác	Implement TTL 30s; Reload gọi force=true.
Kết quả mong đợi	Mount lại trong 30s có thể dùng cache; Reload tạo request mới.
Cách kiểm tra	Network + console; so sánh số request.
Lỗi thường gặp	Nút Reload chỉ gọi lại hook nhưng không bypass cache → sinh viên tưởng reload hỏng.

Bước 22. Tạo phiên bản Axios để so sánh

Trạng thái sau bước: V9 – Axios awareness

```
// src/api/apiClient.js  
import axios from 'axios';  
export const apiClient = axios.create({ timeout: 5000, headers: { Accept: 'application/json' } });  
  
// src/api/orchidService.axios.example.js  
import { apiClient } from './apiClient';  
export async function getOrchidsByAxios() {  
  const response = await apiClient.get('/orchids.json');  
  return response.data;  
}
```

Điểm	Fetch	Axios
Cài package	Không	Có
JSON	response.json()	response.data
HTTP 4xx/5xx	Phải check response.ok	Promise reject mặc định
Interceptors	Không built-in	Có
Mục tiêu Lab	Core extension	So sánh/mở rộng
Mục	Nội dung	
Mục tiêu	Hiểu khác biệt API surface thay vì thay toàn project.	
Thao tác	Tạo file ví dụ, chạy thử nếu giảng viên yêu cầu.	
Kết quả mong đợi	Sinh viên giải thích được response vs response.data.	
Cách kiểm tra	Tạm đổi service import để thử rồi revert về Fetch baseline.	
Lỗi thường gặp	Cài Axios nhưng vẫn dùng <code>await response.json()</code> như Fetch.	

Bước 23. Thêm search/filter (mở rộng sau CORE PASS)

Trạng thái sau bước: V10 – Derived view

```
// Ý tưởng state trong Orchids.jsx
const [keyword, setKeyword] = useState("");
const [category, setCategory] = useState('ALL');
const [specialOnly, setSpecialOnly] = useState(false);

const visibleOrchids = orchids.filter((o) => {
  const matchName = o.orchidName.toLowerCase().includes(keyword.trim().toLowerCase());
  const matchCategory = category === 'ALL' || o.category === category;
  const matchSpecial = !specialOnly || o.isSpecial;
  return matchName && matchCategory && matchSpecial;
});
```

Đây là derived data từ state hiện có; không cần useEffect để tính lại danh sách lọc.

Mục	Nội dung
Mục tiêu	Củng cố state vs derived state.
Thao tác	Tạo SearchBox/CategoryFilter hoặc inline Form; render visibleOrchids thay vì orchids.
Kết quả mong đợi	Filter không tạo request network mới.
Cách kiểm tra	Network phải không tăng khi gõ/filter.
Lỗi thường gặp	Dùng useEffect + setFilteredOrchids gây state trùng lặp không cần thiết.

Bước 24. Break-It Lab: tạo lỗi có chủ đích

Trạng thái sau bước: V10 – Debug evidence

Bug	Cách tạo	Dấu hiệu	Sinh viên phải tìm
404 JSON	Đổi `/orchids.json` → `/orchidss.json`	Error UI, Network 404	URL + response.ok
Invalid JSON	Thêm trailing comma vào orchids.json	Parse error	Response body + Console
Modal null	Bỏ guard orchid?.	Runtime error khi modal chưa có dữ liệu	Null-safe rendering
Effect loop	Cho dependency thay đổi mỗi render	Request lặp	Dependency array
Cache reload sai	Reload không force	Không có network request mới	Cache policy
Mục	Nội dung		
Mục tiêu	Biến lỗi thành evidence học tập.		
Thao tác	Mỗi nhóm chọn ít nhất 2 bug; ghi Symptoms → Hypothesis → Evidence → Fix.		
Kết quả mong đợi	Có debug log hoặc ảnh Network/Console.		
Cách kiểm tra	Sau khi sửa, chạy regression test lại.		
Lỗi thường gặp	Dùng AI trước khi thu thập evidence làm mất mục tiêu debugging.		

Bước 25. Build production và Git checkpoint

Trạng thái sau bước: V10 – Build verified

```
npm run build
npm run preview
```

12. Kiểm tra build không lỗi.
13. Mở preview URL; test list/modal/fetch.
14. git status.
15. git add .
16. git commit -m "lab02 orchid gallery fetch cache complete".

Mục	Nội dung
Mục tiêu	Xác nhận code không chỉ chạy ở dev mode.
Thao tác	Build + preview + commit.
Kết quả mong đợi	dist được tạo; preview chạy; Git sạch sau commit.
Cách kiểm tra	Test lại core functions trên preview.
Lỗi thường gặp	Asset path bắt đầu bằng đường dẫn sai có thể chỉ lộ ở deployment khác base path.

Bước 26. README, Evidence Package và Human Verification Gate

Trạng thái sau bước: FINAL – Ready to submit

- README: mục tiêu, prerequisites, install, run, build, cấu trúc project, data source, cache policy.
- Screenshot 1: Orchid list + NavBar.
- Screenshot 2: Modal detail.
- Screenshot 3: DevTools request `/orchids.json` 200.
- Screenshot 4: Error state khi cố tình 404.
- Git commit hash hoặc link repo nếu dùng.
- AI log: prompt → kết quả → phần đã kiểm chứng → phần đã sửa.

Mục	Nội dung
Mục tiêu	Đóng gói bằng chứng kỹ thuật chứ không chỉ screenshot đẹp.
Thao tác	Hoàn tất README và evidence; trả lời Human Gate ở Phần 12.
Kết quả mong đợi	Có thể giải thích end-to-end flow không nhìn code.
Cách kiểm tra	Bạn học hỏi ngẫu nhiên một câu Human Gate; mở code/Network để chứng minh.
Lỗi thường gặp	README chỉ có `npm run dev` nhưng không nói data flow/cache/error handling.

8. ĐÁNH GIÁ KẾT QUẢ HOÀN THÀNH TỪNG BƯỚC

Bước	Kết quả mong đợi	Cách kiểm tra	Trạng thái
1–2	Project tạo/mở/chạy được	npm run dev + hot reload	
3–5	Bootstrap/Axios cài đúng, CSS hoạt động	package.json + UI	
6–8	Cấu trúc + data + asset	Project tree + data length	
9	NavBar hoạt động	Responsive navbar	
10–13	Card/List/Modal core PASS	Click Detail; close modal	
14–17	Async state model	Spinner → data; retry path	

Bước	Kết quả mong đợi	Cách kiểm tra	Trạng thái
18–20	Fetch HTTP hoạt động	Network /orchids.json = 200	
21	Cache TTL + force reload	Network request count	
22	Axios comparison	Giải thích response.data	
23	Optional filter	Không phát sinh request mới	
24	Debug evidence	Có hypothesis/evidence/fix	
25–26	Build/Git/README/evidence	npm run build + submission package	

9. CHẠY VÀ KIỂM TRA ỨNG DỤNG HOÀN CHỈNH

9.1 Lệnh chạy

```
npm install
npm run dev
# trước khi nộp
npm run build
```

9.2 Test matrix

ID	Chức năng	Thao tác	Kết quả mong đợi
T01	Startup	npm run dev	Không lỗi blocking
T02	Navbar	Mở app	Navbar + brand + menu hiển thị
T03	Bootstrap	Quan sát Button/Card	Có style Bootstrap
T04	List	Chờ load	8 Orchid cards hiển thị
T05	Card fields	Quan sát card	image/name/category/Detail
T06	Detail	Click card 1	Modal mở đúng orchid 1
T07	Different detail	Đóng, click card 5	Modal đổi đúng orchid 5
T08	Close	Click Close/X	Modal đóng
T09	Loading	Hard reload	Spinner xuất hiện trong khi chờ
T10	Fetch	Network	GET /orchids.json
T11	Status	Network details	200 OK
T12	Response	Network Response	JSON array
T13	Error 404	Sai path tạm thời	ErrorMessage xuất hiện
T14	Retry	Sửa path + Try Again	List trở lại
T15	Reload	Click Reload	Có force request mới
T16	Cache	Remount/reload logic trong TTL	Giải thích cache hit/miss
T17	Empty	Tạm orchids.json = []	Hiện empty state

ID	Chức năng	Thao tác	Kết quả mong đợi
T18	No console errors	Console	Không error nghiêm trọng
T19	Responsive	Đổi width	1/2/4 columns hợp lý
T20	Build	npm run build	Build thành công
T21	Preview	npm run preview	Core functions chạy
T22	Optional search	Nhập keyword	Danh sách lọc đúng
T23	Optional category	Chọn category	Filter đúng
T24	Optional special	Toggle special	Chỉ hiện special
T25	AI verification	Chọn 1 prompt AI	Sinh viên chỉ ra bằng chứng kiểm chứng

10. LỖI THƯỜNG GẶP VÀ CÁCH XỬ LÝ

Dấu hiệu	Triệu chứng	Nguyên nhân	Cách xử lý	Phòng tránh
Unsupported Node/Vite	npm create vite@latest cảnh báo engine	Node 21 không thuộc dải Vite hiện hành	Pin Vite 5 hoặc dùng Node 22.12+	Kiểm tra node -v trước scaffold
Bootstrap không style	Card/Button trông như HTML thường	Chưa import CSS	Import bootstrap/dist/css/bootstrap.min.css trong main.jsx	Smoke test Button sau cài
Image 404	Ảnh vỡ	Dùng /public/images	Dùng /images/...	Mở URL asset trực tiếp
Modal crash	Cannot read properties of null	Render orchid khi null	Optional chaining/conditional render	Test modal initial closed
Infinite fetch	Network request lặp	Effect dependencies thay đổi	useCallback + dependency đúng	Đọc Network trước khi sửa
Fetch 404	Error UI	Sai /orchids.json	Đặt file trong public và dùng /orchids.json	Test URL trực tiếp
Fetch 404 không catch	UI parse sai	Không check response.ok	Throw khi !response.ok	Nhớ fetch chỉ reject lỗi network
Axios data undefined	Không render list	Dùng response thay vì response.data	return response.data	So sánh API Fetch/Axios
Loading không tắt	Spinner mãi	Thiếu finally	setLoading(false) trong finally	Test cả success/error
Reload không request	Network không đổi	Cache hit	reload(force=true)	Định nghĩa rõ normal load vs force reload
JSON parse error	Unexpected token	JSON không chuẩn	Double quotes, no comments/trailing comma	Mở file qua browser/validator
Map key warning	Console warning	id thiếu/trùng	Đảm bảo id unique	Kiểm tra dataset trước UI

11. PHẦN MỞ RỘNG VỚI AI ASSISTANT

Nguyên tắc: AI có thể sinh code hoặc giả thuyết, nhưng sinh viên phải đưa ra bằng chứng từ source, Console, Network hoặc test case. Không chấp nhận câu trả lời “AI nói như vậy” làm evidence.

Nhóm	Prompt mẫu
Giải thích	Giải thích data flow từ App → Orchids → useOrchids → orchidService → fetch → state → render.

Nhóm	Prompt mẫu
Giải thích	Giải thích vì sao selectedOrchid nên ở Orchids thay vì mỗi OrchidCard.
Giải thích	Giải thích fetch 404 khác lỗi network như thế nào.
Giải thích	Giải thích loading/error/data như một state machine đơn giản.
Giải thích	Giải thích vai trò useCallback trong useOrchids.
Giải thích	So sánh Fetch response.json() và Axios response.data trong project này.
Giải thích	Giải thích cache hit, cache miss, TTL và stale data bằng timeline 0–60s.
Giải thích	Giải thích vì sao public/orchids.json dùng URL /orchids.json.
Giải thích	Giải thích workflow khi bấm Detail và khi đóng Modal.
Giải thích	Giải thích vì sao search/filter local không cần gọi API mới.
Tối ưu	Review cấu trúc component và chỉ đề xuất refactor nếu giảm coupling rõ ràng.
Tối ưu	Đề xuất cách tách status UI nhưng không đưa Redux/React Query vào core.
Tối ưu	Thiết kế cache invalidation rõ ràng cho nút Reload.
Tối ưu	Đề xuất AbortController cho Fetch và giải thích cleanup.
Tối ưu	Đề xuất debounce cho search nhưng giữ filter hiện tại để hiểu cho sinh viên.
Tối ưu	Review accessibility của Navbar/Card/Modal/image alt.
Tối ưu	Review error messages để phân biệt network, HTTP và parse error.
Tối ưu	Đề xuất test case boundary cho cache TTL 30s.
Tối ưu	Chuẩn bị service để sau này đổi base URL sang Spring Boot.
Tối ưu	Đề xuất pagination nhưng không triển khai nếu chưa cần.
Workflow	Mô tả timeline khi component mount lần đầu.
Workflow	Mô tả timeline khi cache còn hiệu lực.
Workflow	Mô tả timeline khi cache hết hạn.
Workflow	Mô tả timeline khi request trả 404.
Workflow	Mô tả timeline khi JSON invalid.
Workflow	Mô tả toàn bộ state transition khi bấm Reload.
Workflow	Mô tả props/event flow giữa Orchids và OrchidCard.
Workflow	Mô tả Modal detail flow và vì sao cần selectedOrchid.
Workflow	Nếu thay JSON file bằng Spring Boot GET /api/orchids thì file nào đổi?
Workflow	Nếu thêm Router /orchids/:id thì Modal và route detail khác nhau thế nào?
Debug	Danh sách không hiển thị; yêu cầu AI đưa checklist evidence trước khi đưa fix.
Debug	Modal không mở; kiểm tra callback/onDetail/state theo thứ tự.

Nhóm	Prompt mẫu
Debug	/orchids.json 404; phân tích public path và base path.
Debug	Loading không tắt; kiểm tra finally và rejected promise.
Debug	Reload không phát request; phân tích cache force flag.
Nâng cao	Thêm json-server CRUD cho Orchids nhưng giữ Fetch list hiện tại.
Nâng cao	Thêm React Router /orchids và /orchids/:id như optional extension.
Nâng cao	Thêm SWR/React Query và so sánh với cache module thủ công.
Nâng cao	Thêm AbortController để hủy request khi unmount.
Nâng cao	Thiết kế API Spring Boot tương lai: GET /api/orchids và GET /api/orchids/{id}.

11.1 AI Verification Log

Prompt	AI đề xuất	Evidence đã kiểm tra	Quyết định của sinh viên	Bài học

12. CHECKLIST Củng cố SLOT 9 VÀ CHUẨN BỊ LAB 02

Nội dung	Đạt	Chưa đạt	Evidence
Cài được React Bootstrap + Bootstrap	☐	☐	
Import Bootstrap CSS	☐	☐	
Tạo NavBar	☐	☐	
Tạo OrchidsData	☐	☐	
Render Card list	☐	☐	
Modal detail đúng selected Orchid	☐	☐	
Hiểu Promise	☐	☐	
Dùng async/await	☐	☐	
Dùng Fetch	☐	☐	
Phân biệt Fetch và Axios	☐	☐	
Có loading state	☐	☐	
Có error state	☐	☐	

Nội dung	Đạt	Chưa đạt	Evidence
Có empty/data state	☐	☐	
Tách orchidService	☐	☐	
Hiểu service layer	☐	☐	
Có cache TTL	☐	☐	
Hiểu stale cache	☐	☐	
Force Reload bypass cache	☐	☐	
Đọc Network request/response	☐	☐	
Không gọi API trực tiếp trong render	☐	☐	
Giải thích useEffect dependency	☐	☐	
Build production thành công	☐	☐	
Có README/evidence	☐	☐	
Sẵn sàng làm Lab 02/Assignment 01	☐	☐	
AI output đã được kiểm chứng	☐	☐	

12.1 Human Verification Gate – 20 câu

1. Vì sao Lab 02 core nên chạy với static data trước khi chuyển sang Fetch?
2. Props nào đi từ Orchids xuống OrchidCard?
3. Event nào đi từ OrchidCard lên Orchids?
4. selectedOrchid và show khác nhau về vai trò thế nào?
5. Nếu orchid=null, Modal cần xử lý gì?
6. Promise có ba trạng thái nào?
7. await làm gì với Promise?
8. Tại sao loading phải tắt trong finally?
9. Vì sao fetch 404 không tự vào catch chỉ vì status 404?
10. response.ok dùng để làm gì?
11. public/orchids.json được truy cập bằng URL nào?
12. Service layer giải quyết coupling nào?
13. useOrchids chịu trách nhiệm gì, và không nên chịu trách nhiệm gì?
14. Cache hit khác cache miss thế nào?
15. TTL 30 giây có nghĩa gì?
16. Force Reload khác normal load thế nào?
17. Dùng DevTools chứng minh request thực sự xảy ra bằng cách nào?
18. Axios trả dữ liệu ở đâu?
19. Search/filter local có nên gọi API mới không? Vì sao?
20. Nếu thay JSON file bằng Spring Boot API, component nào lý tưởng không cần đổi?

Gợi ý PASS: trả lời được ít nhất 16/20, trace được một request trong Network và giải thích được một state transition từ Loading → Data hoặc Loading → Error.

13. RUBRIC ĐÁNH GIÁ BÀI TẬP/LAB

Tiêu chí	Trọng số	Minh chứng
Project React chạy được	10%	npm run dev/build
React Bootstrap + CSS đúng	10%	UI + package.json
NavBar đúng yêu cầu	8%	Screenshot
Orchid list bằng Card	12%	UI + source
Modal detail đúng	12%	Demo + state trace
Service + Promise/async-await	12%	orchidService/useOrchids
Fetch hoặc Axios	10%	Network + source
Loading/Error/Data	10%	UI states + break-it test
Cache đơn giản	6%	Network evidence + explanation
Code quality/README/Evidence/AI verification	10%	Repo + docs

Mức Đạt: core Lab 02 chạy, list + Modal đúng. Mức Khá: có service + async state + Fetch/Axios. Mức Tốt: có cache, debug evidence, README rõ. Mức Xuất sắc: extension hợp lý nhưng không phá core, có AI verification và giải thích architecture thuyết phục.

PHỤ LỤC A. SOURCE CODE FINAL – BASELINE KHUYẾN NGHỊ

Baseline cuối nên dùng Fetch + cache module đơn giản; Axios giữ ở file example để so sánh. Điều này làm data flow dễ trace và đúng trọng tâm Slot 9.

```
// Luồng final
App.jsx
-> NavBar.jsx
-> Orchids.jsx
  -> useOrchids.js
    -> orchidService.js
      -> fetch('/orchids.json') + cache TTL
  -> OrchidCard.jsx
  -> OrchidDetailModal.jsx
  -> LoadingSpinner.jsx / ErrorMessage.jsx
```

PHỤ LỤC B. README MẪU

```
# Orchid Gallery SPA – Lab 02 + Slot 9

## Requirements
- Node 21 + Vite 5, OR Node 22.12+ + current Vite

## Install
```bash
npm install
```

## Run
```bash
npm run dev
```

## Build
```

```

```bash
npm run build
```

## Core features
- React Bootstrap Navbar
- Orchid cards
- Detail Modal
- Service layer
- Fetch /orchids.json
- Loading / Error / Empty / Data states
- 30-second cache + force reload

## Evidence
Attach screenshots of list, modal, Network request, and error-state test.

```

PHỤ LỤC C. WORKFLOW TRACE WORKSHEET

| Mốc | Component/Layer | Input | Action | Output/Evidence |
|-----|-----------------|-------------------|--------------------|----------------------------|
| 1 | App | Browser opens app | Render layout | Navbar + Orchids mounted |
| 2 | useOrchids | mount | setLoading(true) | Spinner |
| 3 | orchidService | getOrchids | cache check | cache hit or Fetch |
| 4 | Fetch | /orchids.json | GET | HTTP response |
| 5 | useOrchids | data/error | set state | loading=false |
| 6 | Orchids | state | conditional render | cards/error/empty |
| 7 | OrchidCard | click Detail | onDetail(orchid) | selectedOrchid + show=true |
| 8 | Modal | props | render detail | visible selected Orchid |

PHỤ LỤC D. TÀI LIỆU THAM KHẢO

- Official Lab 02 – Create Single Page Application with ReactJS (theo nội dung giảng viên cung cấp).
- React-Bootstrap Documentation – Getting Started / Introduction.
- Vite Documentation – Getting Started và Node.js compatibility.
- MDN Web Docs – Fetch API, Promise, async/await, HTTP caching.
- Axios Documentation – Introduction.
- React Documentation – useState, useEffect, useCallback.

Ghi chú: tài liệu cố ý không bắt buộc React Router trong Lab 02 core. Router, json-server CRUD, React Query/SWR và Spring Boot API là extension sau khi core đạt PASS.

FINAL SUBMISSION CHECK

| Hạng mục | PASS khi |
|----------------------|--|
| Functional | List, Detail Modal, loading/error/data, reload hoạt động. |
| Lab 02 compatibility | NavBar + React Bootstrap + Orchids component + Card + Modal. |
| Slot 9 evidence | Có Promise/async-await, service layer, Fetch/Axios và cache. |
| Technical | Console sạch lỗi blocking; Network trace hợp lý; build thành công. |

| Hạng mục | PASS khi |
|---------------|---|
| Understanding | Sinh viên qua Human Verification Gate. |
| Documentation | README + screenshots + Git/evidence đầy đủ. |